NoteCad - Voice-First AI Workspace: Master Blueprint

A. Document Control

Field	Value
Document ID	NOTECAD-BP-001
Title	NoteCad - Voice-First AI Workspace: Master Blueprint
Version / Date	1.1 | 14 October 2026
Classification	CONFIDENTIAL - internal product architecture

Purpose. This is the "all-ends" master blueprint for **NoteCad**: a personal, voice-first workspace where a single graph of Nodes is shown as a Mind Map (bubble cloud), an Infinite Whiteboard, a Folder Tree and a Notepad. Each person who uses NoteCad gets their own workspace that the AI shapes around their life or work; the farmer in this document is only the worked example from the brief. It covers the front end, the OS-level floating capture widget, the AI routing engine, the database and real-time layer, a traced end-to-end lifecycle, and a four-phase build plan. It assumes a design system and a few optional platform services, listed in Section 7.

How to use it with Claude. Generate this blueprint first, then build one section per message. The three follow-up prompts in Section 9 are tuned to this document so each step produces working code that matches the schema and naming used here.

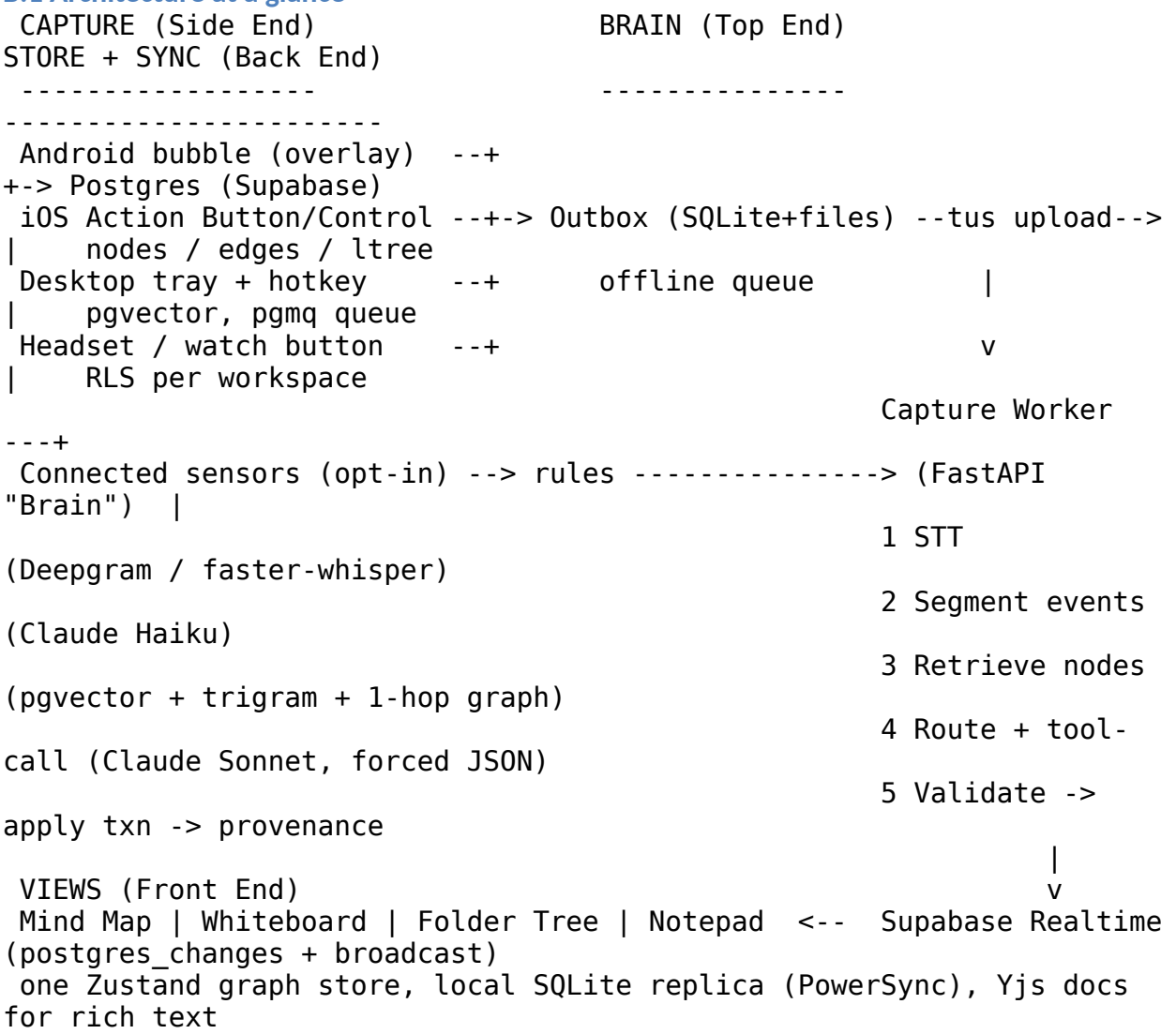
B. Executive Summary

The problem. People who run real operations or busy lives (a farm, a workshop, a field team, a household, a study load) think and report in speech, on the move, often with no signal. Notes apps make them file things; project tools make them type. Information ends up scattered across WhatsApp voice notes, paper and memory.

The product. NoteCad lets the user tap a floating button, say what happened, and put the phone away. The AI transcribes the note, splits it into events, matches each event to the right place in that person's own map, and updates it: a note is appended, a task is created, a bubble turns red. Every view (mind map, whiteboard, folders, notepad) reads the same data, so the change shows up everywhere at once. Nothing is pre-set per industry: a farmer, a mechanic, a student or a manager each start empty, and the onboarding conversation builds their own structure.

Why this stack. NoteCad is built from common building blocks: a design-token system (tokens, typography, logos, a document template), an AI assistant fabric, an optional Edge AI Node for offline / sovereign AI, an on-device privacy pattern, and a provenance record on every change. NoteCad adds a personal capture-and-memory layer on top of them.

B.1 Architecture at a glance



B.2 Key decisions

Area	Decision	Why
Client stack	React + Vite (PWA) wrapped with Capacitor for Android/iOS; Tauri 2 for desktop	Mind-map and whiteboard libraries are DOM-first; one TypeScript codebase for every platform
Views	React Flow (@xyflow/react) + d3-force, tldraw, Tiptap, react-arborist	Mature, React-native APIs, each renders the same Node rows
Data	Postgres + ltree + edges table + pgvector (Supabase)	One structure gives a tree (folders) and a graph (mind map) without a second database
Offline	SQLite outbox + PowerSync	People lose signal; nothing

Area	Decision	Why
	replica + tus resumable uploads	may be lost, everything is idempotent
STT	Deepgram Nova-3 (cloud, keyterm prompting) with faster-whisper on an Edge AI Node as offline fallback / Afrikaans path	Noise-robust, custom vocabulary from each user's own node names
Routing LLM	Claude via tool use with a forced JSON action schema, run through an assistant fabric; local model on an Edge AI Node when offline	Deterministic, validated mutations instead of free text
Interop	NoteCad exposes an MCP server	Assistant agents, Claude Desktop and other MCP clients can read and write a user's nodes, with that user's permission
Personalisation	One private workspace per person; structure, vocabulary and memory learned only from that person	NoteCad is personal: no shared industry template, no cross-user data

1. UI/UX & Front-End Architecture

1.1 One graph, four lenses

Everything is a **Node**. A view is a lens over the same rows, never a copy. The client keeps one normalised graph store (Zustand) fed by a local SQLite replica; each view subscribes to a selector and renders the same node ids.

View	Library	What it reads	What it writes
Mind Map (bubble cloud)	@xyflow/react (React Flow) + d3-force layout worker	nodes, edges (tree + cross-links), node_status, node_layouts[view='mindmap']	x/y pins, collapse state, new links by drag
Infinite Whiteboard	tldraw SDK with a custom NodeShape	same nodes as cards; free shapes stored as kind='sketch'	positions[view='board'], sketches, arrows -> edges
Folder Tree	react-arborist (virtualised)	nodes ordered by ltree path + sort_key	re-parent (drag), rename, reorder
Notepad	Tiptap (ProseMirror) + Yjs	note blocks of the selected node; mentions resolve to	Yjs updates; @mention creates an

View	Library	What it reads	What it writes
		nodes	edge

Seamless switching. The selected node, zoom target and filter are global state, so switching lens keeps context. A shared-element transition (Framer Motion `layoutId=node.id`) morphs a bubble into a folder row or a whiteboard card. A split mode shows Mind Map left and Notepad right; tapping a bubble opens its notes.

Blended mode. On tablet/desktop the whiteboard can host a live Mind Map frame and Notepad frames as tldraw custom shapes, so the user can annotate around a living map. Positions per lens live in `node_layouts`, so moving a bubble does not disturb the folder order.

> **Note:** Licensing check: tldraw production use without its watermark needs a tldraw business licence. React Flow (MIT), Tiptap core (MIT), react-arborist (MIT) and Yjs (MIT) are free. Budget the tldraw licence in Phase 3 or swap in Excalidraw (MIT) if needed.

1.2 Visual status system

A node's look is computed from data, never set by hand. The server keeps `status`, `priority`, open-task and open-alert counts, and a rolled-up `child_worst_status`; the client maps them to visuals using the design system tokens (`--brand-*`), extended with a small `--nc-*` set for the extra status colours.

Status	Colour token	Bubble treatment	Typical trigger
ok	–brand-accent #14b8a6	Solid fill, calm	Default, task closed, alert resolved
info	–brand-primary #6366f1	Thin ring	New note appended in last 24 h
routine_due	–nc-amber #f5a524	Progress arc around bubble	Routine due within its window (e.g. dipping every 6 weeks)
warning	–nc-orange #ff7a1a	Thick ring + badge	Injury, minor fault, overdue routine
critical	–nc-red #e5484d	Red fill + slow pulse (respects reduced-motion)	Breakdown, leak, animal down, safety
pending	–text-muted #8a919e	Dashed ghost outline	Offline capture routed on-device, awaiting server confirm
done	–nc-green #30a46c	Check glyph, fades to ok after 48 h	Milestone completed (e.g. planting finished)

Size. `r = r0 * (1 + 0.25 * log2(1 + descendant_count)) * (1 + 0.15 * open_items)`, clamped. Busy branches grow; quiet ones shrink.

Badges. Top-right shows open tasks plus open alerts. A small "echo dot" on a parent shows the worst status of its children, so `Livestock` shows a red echo dot when one ewe is injured, without turning the whole branch red. Exception: `channel` nodes (e.g. `Health Alerts`, `Maintenance`) mirror their status onto their direct parent, so `Sheep` itself turns red.

Decay. Alerts stay until resolved. Info rings decay after 24 h; done states revert to ok after 48 h. All timers are server-side so every device agrees.

Accessibility. Colour is never the only signal: each status has an icon and a text label, rings have 3:1 contrast against `--surface-bg #0f1115`, and pulse animation is disabled under `prefers-reduced-motion`.

1.3 Cross-platform stack recommendation

Layer	Choice	Notes
Language / build	TypeScript, React 19, Vite, pnpm workspaces, Node 24	One codebase for web, mobile and desktop
Mobile shell	Capacitor (Android + iOS)	Native Kotlin/Swift plugins for overlay bubble, Action Button, background upload
Desktop shell	Tauri 2	Tray icon, global hotkey, always-on-top mini recorder; much smaller than Electron
State	Zustand graph store + TanStack Query for server calls	Selectors per view keep re-renders small
Local DB / sync	SQLite (capacitor-sqlite / wa-sqlite in browser) + PowerSync	Postgres <-> SQLite partial replica, offline writes
Rich text	Tiptap + Yjs (y-indexeddb locally, Hocuspocus or Supabase broadcast for live collab)	CRDT avoids merge conflicts in notes
Styling	Design system tokens (CSS custom properties) + Tailwind mapped to those tokens; Roboto / Roboto Mono	Dark-first (#0f1115), primary-to-accent brand gradient
Testing	Vitest, Playwright (web), Maestro (mobile flows)	Golden tests for routing outputs

1.4 Code layout

```
notecad/
  apps/
```

```
    web/            React + Vite PWA (all four views)            ->
also the Capacitor webDir
    mobile/         Capacitor project: android/, ios/, native plugins
    desktop/        Tauri 2 shell (tray, hotkey, mini recorder)
  packages/
    core/           Node graph types, Zustand store, selectors,
status->visual mapping
    ui/             Design system components (tokens, logos, icons)
    views/          mindmap/, whiteboard/, tree/, notepad/
    sync/           outbox, upload (tus), PowerSync schema, conflict
rules
    schema/         JSON Schemas for AI actions (shared by client +
brain)  <- single source of truth
  services/
    brain/          Python FastAPI capture worker: STT, segmentation,
retrieval, routing, apply
    mcp/            NoteCad MCP server for the assistant fabric
    edge/           Edge AI Node profile (offline STT + local LLM,
sovereign mode)
  supabase/
    migrations/     SQL (Section 4), RLS policies, pgmq queues, cron
jobs
  docs/             This blueprint, ADRs, document metadata
```

2. Floating Widget & Side End (OS Integration, Offline)

2.1 Capture surfaces per platform

Platform	Primary surface	Secondary surfaces	Key constraints
Android	Floating “chat-head” bubble: a WindowManager view with TYPE_APPLICATION_OVERLAY, hosted by a foreground service (type microphone)	Quick Settings tile (TileService), home-screen widget (Jetpack Glance), Bluetooth/headset media button, share-to-NoteCad	SYSTEM_ALERT_WINDOW must be granted by the user in Settings. Start the microphone foreground service while the app is visible (when the user turns the bubble on) and keep it alive with a persistent notification; only open AudioRecord on tap. Starting a mic service later from the background is restricted on Android 14+.
iOS	Action Button / Control Center &	Lock Screen + Home widgets (WidgetKit	No system overlays on iOS. Recording

Platform	Primary surface	Secondary surfaces	Key constraints
	Lock Screen control (iOS 18 ControlWidget) running an AudioRecordingIntent	deep links), Live Activity / Dynamic Island showing recording timer and routing result, Siri / Shortcuts phrase “NoteCad note”	without opening the app requires the AudioRecordingIntent path, which must show a Live Activity while recording. Action Button exists only on newer iPhones, so always offer the Control and widget too.
Desktop (Win/macOS/Linux)	Tauri tray icon + global hotkey (default Ctrl+Shift+Space): hold-to-talk mini window, always on top	Drag a file or screenshot onto the tray to attach to the last-routed node	Ask microphone permission once; on macOS add NSMicrophoneUsageDescription
Wearables & accessories	Bluetooth headset / earbud button (media-button press starts a capture)	Wear OS tile and watchOS complication with hold-to-talk	Each person picks the surfaces they want during onboarding; none are required

Android bubble - skeleton

```
class BubbleService : Service() {
  private lateinit var wm: WindowManager
  private lateinit var bubble: View
  override fun onStartCommand(i: Intent?, f: Int, id: Int): Int {
    startForeground(NOTIF_ID, buildNotification(),
        ServiceInfo.FOREGROUND_SERVICE_TYPE_MICROPHONE)          //
started while app is visible
    wm = getSystemService(WINDOW_SERVICE) as WindowManager
    bubble = LayoutInflater.from(this).inflate(R.layout.bubble, null)
    val lp = WindowManager.LayoutParams(WRAP_CONTENT, WRAP_CONTENT,
        WindowManager.LayoutParams.TYPE_APPLICATION_OVERLAY,
        WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE,
PixelFormat.TRANSLUCENT)
    bubble.setOnTouchListener(DragAndHoldToTalk(lp,
        onStart = { Recorder.start(this) },                      //
Opus 16 kHz mono -> outbox file
        onStop  = { Recorder.stopAndEnqueue(this) }))            //
writes outbox row, kicks WorkManager
    wm.addView(bubble, lp)
    return START_STICKY
```

```
  }
}
```

iOS Action Button / Control - skeleton

```
struct CaptureNoteIntent: AudioRecordingIntent {          // iOS 18+
  static var title: LocalizedStringResource = "Capture NoteCad note"
  func perform() async throws -> some IntentResult {
    try await LiveActivityController.startRecordingActivity() //
required for background recording
    try await Recorder.shared.start()                          //
writes to App Group container
    return .result()
  }
}
struct CaptureControl: ControlWidget {                     // Control
Center / Lock Screen / Action Button
  var body: some ControlWidgetConfiguration {
    StaticControlConfiguration(kind: "com.example.notecad.capture") {
      ControlWidgetButton(action: CaptureNoteIntent())
{ Label("NoteCad", systemImage: "mic.fill") }
    }
  }
}
```

2.2 Offline-first capture and sync

Rule: a capture is never lost and never applied twice. Every capture gets a UUIDv7 `capture_id` on the device. That id is the idempotency key for upload, transcription, routing and database writes.

1. **Record** Opus 16 kHz mono (Ogg on Android/desktop, CAF/M4A on iOS, transcoded server-side) to app storage. Store GPS fix, battery, device id, local time and SHA-256.
2. **Outbox row** in SQLite: `captures_outbox(capture_id, file_path, sha256, bytes, created_at, lat, lon, accuracy_m, lang_hint, state, attempts, last_error)`.
3. **Provisional pass (optional, on device)**: whisper.cpp base model or the platform speech recogniser produces a rough transcript; a local keyword matcher against cached node aliases marks matching bubbles as `pending` (dashed ghost) so the user sees something happen even offline.
4. **Upload** with tus resumable uploads to Supabase Storage (`captures/{workspace}/{capture_id}.ogg`) in 1 MB chunks; resume after drop-outs. Android: WorkManager with a CONNECTED constraint and exponential backoff. iOS: background URLSession plus BGProcessingTask. Desktop: in-process retry.
5. **Register** `POST /captures` with the metadata; the server inserts with `ON CONFLICT (capture_id) DO NOTHING` and enqueues a pgmq job.
6. **Track** the state machine through Realtime broadcast `capture:{id}`: queued -> uploaded -> transcribing -> routing -> applied | needs_review | failed.

7. **Reconcile**: when the server result arrives, provisional ghosts are replaced by real status; mismatches are shown in the Inbox for one-tap correction.

Data	Conflict rule
Rich text notes	Yjs CRDT merge - no conflicts
Node fields (title, status, attributes)	Per-field last-writer-wins on a hybrid logical clock; a user edit always beats an AI edit made earlier than the user saw it
Structure (re-parent)	Server validates no cycles; on conflict the later move wins and the earlier one is recorded in the audit log
AI actions	Applied only on the server inside one transaction keyed by capture_id; replays are no-ops

2.3 Offline and sovereign mode with an Edge AI Node

Some users work where the internet is unreliable, or want their data to stay in their own building. For them, NoteCad pairs with an **Edge AI Node**, an offline-first on-premises AI box (local models, data in-house, cloud optional). Phones on the same Wi-Fi discover the node via mDNS (`_notecad._tcp`). When the node is reachable and the cloud is not, or when the user has chosen sovereign mode, the outbox uploads to the node, which runs faster-whisper STT and a local model for routing and keeps its own replica of that user's workspace. In hybrid mode it forwards the capture and result to the cloud later; the cloud re-validates and accepts the result when the node graph version matches, otherwise it re-routes. In sovereign mode nothing leaves the node.

3. AI Engine & Top End (Memory, Learning, Routing)

3.1 Voice-to-text for noisy outdoor audio

Engine	Where	Strengths	Use in NoteCad
Deepgram Nova-3	Cloud, streaming + batch	Strong in noise; keyterm prompting for custom vocabulary; word timings and confidences	Primary for English when online. Keyterms are generated from the user's node titles and aliases (e.g. "Dohne Merino", "John Deere 6155M", "south paddock")
faster-whisper large-v3-turbo	Edge AI Node (GPU/CPU) or a cloud worker	Multilingual incl. Afrikaans and code-switching; runs offline; initial_prompt carries vocabulary	Afrikaans path, offline / sovereign mode, fallback when Deepgram confidence < 0.75

Engine	Where	Strengths	Use in NoteCad
whisper.cpp (tiny/base)	On device	Free, offline, fast enough for 30 s clips	Provisional transcript only (pending ghosts)

Pre-processing: Silero VAD trims silence and wind gaps; DeepFilterNet removes engine and wind noise before STT; loudness is normalised to -20 LUFS. **Language:** detect per capture; the user profile stores preferred languages (Afrikaans, English). Confirm Deepgram's Afrikaans coverage before relying on it; until then, notes detected as `af` go to Whisper.

Vocabulary loop: every time a node is created or an alias is learned, the keyterm list is rebuilt (top 100 by recency and frequency). This is the single biggest accuracy lever for farm terms, brand names and paddock names.

3.2 Routing pipeline

Stage	What happens	Model / tech	Output
1 Normalise	Fix numbers and units ("two hundred hectares" -> 200 ha), strip fillers	Rules + small LLM	clean transcript
2 Segment	Split into independent events; one event = one thing that happened or one request	claude-haiku-5-5	events[] with text spans
3 Retrieve	Per event: hybrid search over nodes (pgvector cosine on path+title+aliases+summary, pg_trgm on aliases), top 8; expand 1 hop via edges; always include the ontology spine (depth <= 2)	Postgres + embeddings (bge-m3 or voyage, 1024-d)	candidate node set with ids + paths
4 Route	LLM receives the spine, candidates, profile facts, routines and the event; must call `apply_capture_actions`	claude-sonnet-5-5, forced tool choice; Ollama model on Edge offline	actions[] (JSON Schema 3.5)
5 Validate	Schema check; node ids must exist in candidates or be created earlier in the	Python (pydantic + jsonschema)	accepted / needs_review

Stage	What happens	Model / tech	Output
	same batch; no cycles; confidence gates		
6 Apply	One transaction per capture: notes, nodes, tasks, alerts, edges, status rollup, provenance	Postgres function `nc_apply_actions`	audit rows + Realtime events
7 Ask-back	Low confidence or ambiguous target -> Inbox card with 2-3 suggested routes; the user's tap becomes training data (alias + correction)	UI + `route_corrections` table	learned alias

Decision policy: append vs create vs alert

Situation	Action	Rule
Event clearly about an existing node	append_note	Best candidate similarity >= 0.80 and the LLM agrees
About a new instance of a known type (a new ewe, a new pump)	create_node under the type's parent	Create only if no candidate >= 0.65 and the parent exists. Never create top-level categories without ask_user
Something is broken, injured, leaking, missing, unsafe	set_status (warning/critical) + create_alert	Severity from wording and asset criticality in the profile
Something needs doing	create_task	Due date from wording, else from routine, else none. Priority from severity
Progress or completion ("finished planting")	update_attributes + set_status done	Numeric attributes only change with an explicit number in the note
Location or relation mentioned ("near the bean fields")	link_nodes	Edge type located_at / affects / related_to
Durable fact about the user's operation	upsert_profile_fact	Only stable facts (herd size, crop area), never one-off events
Unsure (confidence < 0.6 or two close candidates)	ask_user	Nothing is written except an Inbox item

3.3 Long-term learning (memory model)

Memory	Stored in	Updated by	Used for
Ontology graph	nodes, edges (ltree paths)	create_node, user edits, onboarding interview	Spine + candidates in every routing call
Profile facts	profile_facts (key, value, source, confidence)	upsert_profile_fact, onboarding	Industry context, units, languages, asset criticality
Aliases	node_aliases (alias, lang, weight)	User corrections, repeated phrasing	STT keyterms, trigram match ("die groot pomp" -> Main Water Pump)
Episodic notes	notes + note embeddings	Every applied capture	RAG answers: "when did the tractor last break?"
Routines and seasons	routines (rrule, window, node_id)	Nightly pattern miner, user approval	routine_due status, reminders, seasonal context (planting Oct-Dec)
Node summaries	nodes.summary	Nightly consolidation	Short context instead of full note history

Nightly consolidation (pg_cron -> brain job): for each workspace, summarise changed nodes, mine recurring events (same node + same verb at regular intervals) into proposed routines, decay stale info statuses, and rebuild the keyterm list. Proposals appear in the Inbox; nothing structural changes without the user's approval.

Personal onboarding interview: a 3-minute voice conversation ("Tell me about your work, your week and what you look after") seeds each person's own spine; there are no fixed industry templates. Optional starter packs (Farm, Workshop, Office, Study, Household) only pre-fill suggestions the user can accept or delete. For the farming example from the brief it proposes `Operations > Livestock > {Cattle, Sheep}`, `Operations > Crops > Beans (3,000 ha)`, `Operations > Vehicles > {Tractors, Bakkies}`, `Operations > Infrastructure > {Water, Fencing, Buildings}`, `Admin > {Finance, Staff, Compliance}`. A student would instead get `Courses`, `Assignments`, `Projects`, `Personal`.

3.4 System prompt (routing)

```
You are the NoteCad Router.
You turn ONE event from a user's voice note into database actions on
their workspace graph.

CONTEXT YOU RECEIVE
- <profile>: stable facts about the user and their operation
(industry, units, languages, asset criticality).
- <spine>: the top two levels of their node tree, as "path | node_id".
- <candidates>: up to 12 nodes likely related to this event, each with
```

```
node_id, path, kind,
  status, aliases, summary, attributes and similarity score.
- <routines>: active routines that may relate to this event.
- <event>: the event text, the full transcript for context, capture
time, GPS, language.

RULES
1. Always respond by calling apply_capture_actions exactly once. Never
answer in prose.
2. Use only node_ids that appear in <spine> or <candidates>, or
temp_ids you create in this call.
3. Prefer appending to an existing node. Create a node only for a new
specific instance
   (one ewe, one pump) under an existing parent. Never create top-
level categories; use ask_user.
4. Anything broken, injured, leaking, missing, sick or unsafe ->
set_status + create_alert.
   critical = safety risk, animal suffering, water/power loss, or a
stopped critical machine.
   warning  = degraded but working, minor injury, overdue routine.
5. Anything that needs doing -> create_task. Infer due dates only from
explicit words
   ("tomorrow", "before Friday") or a matching routine; otherwise
leave due_at null.
6. Change numeric attributes only when the user states a number. Keep
units from <profile>.
7. If the event names a place or affected asset, add link_nodes
(located_at, affects, related_to).
8. Record durable facts with upsert_profile_fact (herd size, hectares,
equipment owned). Not events.
9. Give every action a confidence 0-1. If the best target is below
0.6, or two targets are within
   0.1 of each other, emit ask_user with up to 3 options instead of
writing.
10. Write note text in the user's language, as a clean third-person
log line, keeping all facts.
    Do not invent facts, names, causes or quantities.
11. Keep evidence: each action quotes the exact transcript span it
came from.
```

3.5 Tool definition and JSON Schema

The schema lives in `packages/schema/actions.schema.json` and is shared by the brain (validation), the client (rendering previews) and the MCP server.

```
{
  "name": "apply_capture_actions",
  "description": "Apply the actions extracted from one event of a
voice capture to the NoteCad graph.",
  "input_schema": {
    "type": "object",
```

```
    "additionalProperties": false,
    "required": ["capture_id", "event_index", "summary", "actions"],
    "properties": {
      "capture_id": { "type": "string", "format": "uuid" },
      "event_index": { "type": "integer", "minimum": 0 },
      "summary": { "type": "string", "maxLength": 200 },
      "actions": {
        "type": "array",
        "minItems": 1,
        "maxItems": 12,
        "items": { "$ref": "#/$defs/action" }
      }
    },
    "$defs": {
      "nodeRef": {
        "type": "string",
        "description": "Existing node_id (uuid) or a temp_id like
'tmp:1' created earlier in this call"
      },
      "base": {
        "type": "object",
        "required": ["type", "confidence", "evidence"],
        "properties": {
          "type": { "type": "string" },
          "confidence": { "type": "number", "minimum": 0, "maximum": 1
},
          "evidence": { "type": "string", "description": "Exact
transcript span" }
        }
      },
      "action": {
        "oneOf": [
          { "allOf": [ { "$ref": "#/$defs/base" }, {
            "properties": {
              "type": { "const": "append_note" },
              "node_id": { "$ref": "#/$defs/nodeRef" },
              "text": { "type": "string" },
              "tags": { "type": "array", "items": { "type": "string" }
}
            },
            "required": ["node_id", "text"] } ] },
          { "allOf": [ { "$ref": "#/$defs/base" }, {
            "properties": {
              "type": { "const": "create_node" },
              "temp_id": { "type": "string", "pattern": "^tmp:[0-9]+$"
},
              "parent_id": { "$ref": "#/$defs/nodeRef" },
              "title": { "type": "string", "maxLength": 80 },
              "kind": { "enum": ["category", "channel", "asset",
"animal", "group", "field", "place", "person", "project", "topic"] },
```

```
              "aliases": { "type": "array", "items": { "type":
"string" } },
              "attributes": { "type": "object" }
            },
            "required": ["temp_id", "parent_id", "title", "kind"] } ]
},
          { "allOf": [ { "$ref": "#/$defs/base" }, {
            "properties": {
              "type": { "const": "set_status" },
              "node_id": { "$ref": "#/$defs/nodeRef" },
              "status": { "enum": ["ok", "info", "routine_due",
"warning", "critical", "done"] },
              "reason": { "type": "string" }
            },
            "required": ["node_id", "status", "reason"] } ] },
          { "allOf": [ { "$ref": "#/$defs/base" }, {
            "properties": {
              "type": { "const": "create_alert" },
              "node_id": { "$ref": "#/$defs/nodeRef" },
              "severity": { "enum": ["warning", "critical"] },
              "title": { "type": "string", "maxLength": 120 },
              "category": { "enum": ["health", "breakdown", "leak",
"safety", "security", "loss", "other"] }
            },
            "required": ["node_id", "severity", "title", "category"] }
] },
          { "allOf": [ { "$ref": "#/$defs/base" }, {
            "properties": {
              "type": { "const": "create_task" },
              "node_id": { "$ref": "#/$defs/nodeRef" },
              "title": { "type": "string", "maxLength": 120 },
              "priority": { "enum": ["low", "normal", "high",
"urgent"] },
              "due_at": { "type": ["string", "null"], "format": "date-
time" },
              "assignee_hint": { "type": ["string", "null"] }
            },
            "required": ["node_id", "title", "priority"] } ] },
          { "allOf": [ { "$ref": "#/$defs/base" }, {
            "properties": {
              "type": { "const": "update_attributes" },
              "node_id": { "$ref": "#/$defs/nodeRef" },
              "set": { "type": "object" },
              "increment": { "type": "object", "additionalProperties":
{ "type": "number" } }
            },
            "required": ["node_id"] } ] },
          { "allOf": [ { "$ref": "#/$defs/base" }, {
            "properties": {
              "type": { "const": "link_nodes" },
```

```
              "src_id": { "$ref": "#/$defs/nodeRef" },
              "dst_id": { "$ref": "#/$defs/nodeRef" },
              "edge_type": { "enum": ["located_at", "affects",
"related_to", "part_of", "assigned_to"] }
            },
            "required": ["src_id", "dst_id", "edge_type"] } ] },
          { "allOf": [ { "$ref": "#/$defs/base" }, {
            "properties": {
              "type": { "const": "upsert_profile_fact" },
              "key": { "type": "string" },
              "value": {}
            },
            "required": ["key", "value"] } ] },
          { "allOf": [ { "$ref": "#/$defs/base" }, {
            "properties": {
              "type": { "const": "ask_user" },
              "question": { "type": "string" },
              "options": {
                "type": "array", "maxItems": 3,
                "items": { "type": "object", "required": ["label",
"node_id"],
                  "properties": { "label": { "type": "string" },
"node_id": { "$ref": "#/$defs/nodeRef" } } }
              }
            },
            "required": ["question"] } ] }
        ]
      }
    }
  }
}
```

Example: sheep injury + tractor hose (scenario from the brief)

Transcript: *"One of the ewes in the south paddock broke her back left leg, and the John Deere tractor's hydraulic hose just burst near the bean fields."* Segmentation yields two events. Router output for event 1 (sheep):

```
{
  "capture_id": "0192f6a4-7c1e-7b3a-9d2e-5a1f0c3b8e11",
  "event_index": 0,
  "summary": "Ewe in south paddock has a broken back left leg",
  "actions": [
    { "type": "append_note", "node_id": "6b1c0e2a-1d4f-4a8e-9b7c-
2f3e4d5a6b01",
      "text": "Ewe in the south paddock broke her back left leg.",
      "tags": ["injury", "leg"], "confidence": 0.93,
      "evidence": "One of the ewes in the south paddock broke her back
left leg" },
    { "type": "set_status", "node_id": "6b1c0e2a-1d4f-4a8e-9b7c-
2f3e4d5a6b01",
```

```
      "status": "critical", "reason": "Animal injury: broken leg",
      "confidence": 0.9, "evidence": "broke her back left leg" },
    { "type": "create_alert", "node_id": "6b1c0e2a-1d4f-4a8e-9b7c-
2f3e4d5a6b01",
      "severity": "critical", "title": "Ewe with broken back left leg
(south paddock)",
      "category": "health", "confidence": 0.9, "evidence": "broke her
back left leg" },
    { "type": "create_task", "node_id": "6b1c0e2a-1d4f-4a8e-9b7c-
2f3e4d5a6b01",
      "title": "Call vet / treat ewe with broken leg", "priority":
"urgent", "due_at": null,
      "assignee_hint": null, "confidence": 0.85, "evidence": "broke
her back left leg" },
    { "type": "link_nodes", "src_id": "6b1c0e2a-1d4f-4a8e-9b7c-
2f3e4d5a6b01",
      "dst_id": "9a8b7c6d-5e4f-4321-8a9b-0c1d2e3f4a02", "edge_type":
"located_at",
      "confidence": 0.88, "evidence": "in the south paddock" }
  ]
}
```

Here `6b1c...` is `Operations > Livestock > Sheep > Health Alerts` and `9a8b...` is `Infrastructure > Paddocks > South Paddock`. Event 2 produces the same pattern on `Operations > Vehicles > Tractors > Maintenance` (critical, category breakdown, task urgent) plus `link_nodes` with `located_at` to the `Beans (3,000 ha)` node. `Health Alerts` and `Maintenance` are `channel` nodes: a channel's status is mirrored onto its parent, so the rollup trigger turns the `Sheep` and `Tractors` bubbles red, and higher ancestors (`Livestock`, `Vehicles`) get echo dots.

3.6 Assistant fabric and other inputs (no voice needed)

- **Ask NoteCad:** an assistant fabric (text, code, images, vision and web) becomes "Ask NoteCad" inside the app: questions like "what is still broken?" or "summarise my week" are answered from the user's own nodes and notes via RAG.
- **NoteCad MCP server** (`services/mcp`): tools `search_nodes`, `get_node`, `append_note`, `create_task`, `list_alerts`, `ask_workspace`, scoped to one user's workspace token. Assistant agents, Claude Desktop and other MCP clients can then read and log work on the user's behalf.
- **Connected sensors (opt-in, per user):** any device that can post MQTT or an HTTPS webhook can be bound to a node (`sensor_bindings`). Rules emit deterministic actions (set_status, create_alert) without an LLM, e.g. pressure below 1.2 bar for 5 minutes -> `Main Water Pump` warning. This is how AIoT and predictive-maintenance services can feed a user's NoteCad.

4. Backend, Database & Server Connections

4.1 Why one structure renders as both tree and mind map

- **Tree (folders):** every node has exactly one `parent_id`, mirrored into an `ltree` `path` for fast subtree queries (`path <@ 'ops.livestock'`). The Folder Tree is an ordered walk of this.
- **Graph (mind map):** the mind map draws the same parent links plus extra rows in `edges` (located_at, affects, related_to). Cross-links never change the folder tree.
- **Search and memory:** `pgvector` embeddings on nodes and notes, `pg_trgm` on aliases. No second database at MVP. If graph queries outgrow recursive CTEs, add Apache AGE on self-hosted Postgres or sync edges to Neo4j later, keeping Postgres as the system of record.

4.2 Core schema (PostgreSQL 16 + Supabase)

```
create extension if not exists ltree;
create extension if not exists vector;
create extension if not exists pg_trgm;

create type nc_status   as enum
('done','pending','ok','info','routine_due','warning','critical');
-- declaration order = severity order, so max()/greatest() pick the
worst status
create type nc_kind     as enum
('root','category','channel','asset','animal','group','field',

'place','person','project','topic','sketch');
create type nc_priority as enum ('low','normal','high','urgent');

create table workspaces (
  id          uuid primary key default gen_random_uuid(),
  name        text not null,
  persona     text,                       -- free text from
onboarding: 'farmer', 'student', ...
  locale      text not null default 'en-ZA',
  created_at  timestamptz not null default now()
);

create table memberships (
  workspace_id uuid references workspaces(id) on delete cascade,
  user_id      uuid references auth.users(id) on delete cascade,
  role         text not null default 'owner' check (role in
('owner','editor','viewer')),
  primary key (workspace_id, user_id)
);

create table nodes (
  id            uuid primary key default gen_random_uuid(),
  workspace_id  uuid not null references workspaces(id) on delete
cascade,
```

```
  parent_id     uuid references nodes(id) on delete restrict,
  path          ltree not null,           -- maintained by trigger
from parent chain (label = short id)
  sort_key      text not null default 'm',-- fractional index for
manual ordering
  kind          nc_kind not null default 'topic',
  title         text not null,
  summary       text,
  attributes    jsonb not null default '{}',   --
{"area_ha":3000,"planted_ha":2800,"phase":"planting"}
  status        nc_status not null default 'ok',
  status_reason text,
  status_since  timestamptz not null default now(),
  child_worst   nc_status not null default 'ok',
  open_tasks    int not null default 0,
  open_alerts   int not null default 0,
  embedding     vector(1024),
  version       bigint not null default 1,
  created_by    text not null default 'user',   -- 'user' | 'ai' |
'iot' | 'mcp'
  created_at    timestamptz not null default now(),
  updated_at    timestamptz not null default now()
);
create index nodes_ws_path   on nodes using gist (path);
create index nodes_ws_parent on nodes (workspace_id, parent_id,
sort_key);
create index nodes_embed     on nodes using hnsw (embedding
vector_cosine_ops);

create table node_aliases (
  node_id uuid references nodes(id) on delete cascade,
  alias   text not null,
  lang    text,
  weight  real not null default 1,
  primary key (node_id, alias)
);
create index node_aliases_trgm on node_aliases using gin (alias
gin_trgm_ops);

create table edges (
  id           uuid primary key default gen_random_uuid(),
  workspace_id uuid not null references workspaces(id) on delete
cascade,
  src_id       uuid not null references nodes(id) on delete cascade,
  dst_id       uuid not null references nodes(id) on delete cascade,
  edge_type    text not null check (edge_type in

('located_at','affects','related_to','part_of','assigned_to','mentions
')),
  weight       real not null default 1,
```

```
  created_by   text not null default 'user',
  unique (src_id, dst_id, edge_type)
);

create table node_layouts (            -- per-lens positions; views
never fight over x/y
  node_id   uuid references nodes(id) on delete cascade,
  view      text not null check (view in ('mindmap','board')),
  x real, y real, w real, h real,
  pinned    boolean not null default false,
  collapsed boolean not null default false,
  primary key (node_id, view)
);

create table captures (
  id            uuid primary key,          -- UUIDv7 from the device =
idempotency key
  workspace_id  uuid not null references workspaces(id) on delete
cascade,
  user_id       uuid not null references auth.users(id),
  storage_path  text not null,
  sha256        text not null,
  duration_ms   int,
  recorded_at   timestamptz not null,
  geo           jsonb,                     -- {"lat":-
25.86,"lon":28.19,"acc_m":12}
  source        text not null default 'phone',   -- phone|desktop|
watch|headset|sensor|mcp
  state         text not null default 'queued'
                check (state in
('queued','uploaded','transcribing','routing',
                                 'applied','needs_review','failed')),
  transcript    jsonb,                     -- STT result incl. words +
confidences
  lang          text,
  error         text,
  created_at    timestamptz not null default now()
);

create table capture_events (
  capture_id  uuid references captures(id) on delete cascade,
  idx         int not null,
  text        text not null,
  llm_output  jsonb,                       -- validated
apply_capture_actions input
  confidence  real,
  primary key (capture_id, idx)
);

create table notes (
```

```
  id           uuid primary key default gen_random_uuid(),
  node_id      uuid not null references nodes(id) on delete cascade,
  capture_id   uuid references captures(id),
  body_md      text not null,              -- plain log line /
markdown
  ydoc         bytea,                      -- Yjs state for rich notes
  tags         text[] not null default '{}',
  embedding    vector(1024),
  author       text not null default 'user',
  created_at   timestamptz not null default now()
);

create table tasks (
  id           uuid primary key default gen_random_uuid(),
  node_id      uuid not null references nodes(id) on delete cascade,
  capture_id   uuid references captures(id),
  title        text not null,
  priority     nc_priority not null default 'normal',
  due_at       timestamptz,
  assignee     uuid references auth.users(id),
  done_at      timestamptz,
  created_at   timestamptz not null default now()
);

create table alerts (
  id           uuid primary key default gen_random_uuid(),
  node_id      uuid not null references nodes(id) on delete cascade,
  capture_id   uuid references captures(id),
  severity     nc_status not null check (severity in
('warning','critical')),
  category     text not null,
  title        text not null,
  resolved_at timestamptz,
  created_at   timestamptz not null default now()
);

create table routines (
  id           uuid primary key default gen_random_uuid(),
  node_id      uuid not null references nodes(id) on delete cascade,
  title        text not null,
  rrule        text not null,              -- RFC 5545, e.g.
'FREQ=WEEKLY;INTERVAL=6'
  window_days int not null default 3,
  next_due     timestamptz,
  approved     boolean not null default false
);

create table profile_facts (
  workspace_id uuid references workspaces(id) on delete cascade,
  key          text not null,              -- 'crops.beans.area_ha'
```

```
  value        jsonb not null,
  confidence   real not null default 1,
  source       text not null,              -- 'onboarding' |
'capture:<id>' | 'user'
  updated_at   timestamptz not null default now(),
  primary key (workspace_id, key)
);

create table action_log (                  -- audit + provenance for
every mutation
  id          bigserial primary key,
  workspace_id uuid not null,
  capture_id  uuid,
  actor       text not null,               -- 'ai:claude-sonnet-5-5' |
'user:<uuid>' | 'iot:<sensor>'
  action      jsonb not null,
  provenance  jsonb not null,              -- {"pipeline":"notecad-
brain 1.0","model":"..."}
  created_at  timestamptz not null default now()
);

create table sensor_bindings (             -- opt-in sensors (MQTT /
webhook) -> node rules
  id         uuid primary key default gen_random_uuid(),
  node_id    uuid not null references nodes(id) on delete cascade,
  device_id  text not null,
  metric     text not null,                -- 'pressure_bar'
  rule       jsonb not null                --
{"op":"<","value":1.2,"for_s":300,"status":"warning"}
);
```

Status rollup (sketch)

```
create or replace function nc_rollup(p_node uuid) returns void
language plpgsql as $$
declare
  n record;
  worst nc_status;
begin
  select * into n from nodes where id = p_node;
  while n.parent_id is not null loop
    select greatest(coalesce(max(greatest(c.status, c.child_worst)),
'ok'), 'ok')
      into worst
      from nodes c where c.parent_id = n.parent_id;
    update nodes set child_worst = worst, updated_at = now() where id
= n.parent_id;
    if n.kind = 'channel' then              -- channels mirror status
onto their parent
      update nodes p set status = n.status, status_reason =
n.status_reason
```

```
      where p.id = n.parent_id and p.status < n.status;
    end if;
    select * into n from nodes where id = n.parent_id;
  end loop;
end $$;
```

> **Note:** The enum is declared in severity order (done < pending < ok < info < routine_due < warning < critical), so `max()` and `greatest()` return the worst status and anything below ok is lifted to ok. Phase 1 replaces this loop with a statement-level trigger that recomputes counts and child_worst in one pass.

Row-level security

```
alter table nodes enable row level security;
create policy nodes_member_rw on nodes
  using      (exists (select 1 from memberships m where m.workspace_id
= nodes.workspace_id and m.user_id = auth.uid()))
  with check (exists (select 1 from memberships m where m.workspace_id
= nodes.workspace_id and m.user_id = auth.uid()
                      and m.role in ('owner','editor')));
-- same pattern for edges, notes, tasks, alerts, captures, routines,
profile_facts.
-- The brain worker uses the service role and always filters by
workspace_id itself.
```

4.3 API and real-time connections

Path	Protocol	Purpose
Supabase Auth	OAuth / email OTP / phone OTP	Users, sessions, JWT carrying user id for RLS
PostgREST (`/rest/v1/*`)	REST	CRUD on nodes, edges, notes, tasks from the views (RLS-protected)
Storage + tus (`/storage/v1/upload/resumable`)	HTTP resumable	Audio upload from the outbox
`POST /api/captures`	REST (Edge Function)	Register capture, enqueue `pgmq.send('captures', ...)`; idempotent on id
`WS /api/stream`	WebSocket relay to Deepgram streaming	Live partial transcript while the user speaks (online only)
Brain worker	Python FastAPI + pgmq consumer	STT -> segment -> retrieve -> route -> validate -> `nc_apply_actions()`
Realtime `postgres_changes`	WebSocket	nodes / alerts / tasks changes filtered by workspace_id -> views update instantly

Path	Protocol	Purpose
Realtime broadcast `capture:{id}`	WebSocket	Progress states and partial results for the floating widget and Live Activity
PowerSync	Sync stream	Partial replica of the workspace into device SQLite for offline reads/writes
NoteCad MCP server	MCP (stdio / streamable HTTP)	Assistant fabric, Claude Desktop, other MCP clients (per-user token)
Push	FCM / APNs	Critical alerts to all workspace members

Why not GraphQL at MVP: PostgREST plus Realtime covers the views with less moving parts; the graph shape is resolved client-side from the replica. GraphQL (pg_graphql) can be added later for third-party integrations.

Live bubbles while speaking: with signal, the widget streams audio over `/api/stream`. Every partial transcript runs a cheap alias match on the client; matching bubbles glow (pending). When the final transcript is routed, the real `postgres_changes` events replace the glow, normally within 2-4 seconds of the user letting go.

5. End-to-End Lifecycle Walkthrough

Voice input: *"Just finished planting the last 200 hectares of beans, but the main water pump is leaking badly."* Example user: a farmer on the Highveld (any user's workspace works the same way). Before the note, `Beans` has `area_ha = 3000`, `planted_ha = 2800`, `phase = planting`.

Step 1 - Audio capture (device)


```
{
  "capture_id": "0192f7b2-3a10-7c44-8e21-6b5d9f0a1c77",
  "workspace_id": "1f0e2d3c-4b5a-4968-8776-655443322110",
  "file": "captures/0192f7b2-3a10-7c44-8e21-6b5d9f0a1c77.ogg",
  "codec": "opus", "sample_rate": 16000, "channels": 1, "duration_ms":
6420,
  "sha256":
"9f2c7d0a5b1e4c3f8a6d2e1b0c9f8e7d6c5b4a3928170615f4e3d2c1b0a99887",
  "recorded_at": "2026-10-14T16:42:08+02:00",
  "geo": { "lat": -26.1182, "lon": 28.9365, "acc_m": 9 },
  "source": "phone", "lang_hint": "en-ZA",
  "outbox_state": "queued"
}
```


Step 2 - Speech-to-text result


```
{
  "capture_id": "0192f7b2-3a10-7c44-8e21-6b5d9f0a1c77",
  "engine": "deepgram:nova-3",
```

```
  "keyterms_used": ["beans", "main water pump", "borehole", "centre
pivot", "south paddock"],
  "language": "en",
  "transcript": "Just finished planting the last 200 hectares of
beans, but the main water pump is leaking badly.",
  "confidence": 0.94,
  "words": [
    { "w": "200", "start": 1.92, "end": 2.31, "conf": 0.97 },
    { "w": "hectares", "start": 2.31, "end": 2.80, "conf": 0.95 },
    { "w": "pump", "start": 4.61, "end": 4.95, "conf": 0.93 },
    { "w": "leaking", "start": 5.03, "end": 5.47, "conf": 0.92 }
  ]
}
```


Step 3 - Segmentation and context lookup


```
{
  "events": [
    { "idx": 0, "text": "Just finished planting the last 200 hectares
of beans" },
    { "idx": 1, "text": "the main water pump is leaking badly" }
  ],
  "retrieval": [
    { "idx": 0, "candidates": [
      { "node_id": "c3a1e0f2-7b6d-4c5e-9a8b-1d2e3f4a5b10", "path":
"Operations > Crops > Beans",
        "kind": "field", "attributes": { "area_ha": 3000,
"planted_ha": 2800, "phase": "planting" },
        "score": 0.91 },
      { "node_id": "d4b2f1e3-8c7e-4d6f-8b9c-2e3f4a5b6c20", "path":
"Operations > Crops > Beans > Planting Log",
        "kind": "channel", "score": 0.84 } ] },
    { "idx": 1, "candidates": [
      { "node_id": "e5c3a2f4-9d8f-4e7a-9cad-3f4a5b6c7d30", "path":
"Infrastructure > Water > Main Water Pump",
        "kind": "asset", "aliases": ["main pump", "die groot pomp",
"borehole pump"], "score": 0.95 },
      { "node_id": "f6d4b3a5-0e9a-4f8b-8dbe-4a5b6c7d8e40", "path":
"Infrastructure > Water > Centre Pivot 2",
        "kind": "asset", "score": 0.58 },
      { "node_id": "c3a1e0f2-7b6d-4c5e-9a8b-1d2e3f4a5b10", "path":
"Operations > Crops > Beans",
        "kind": "field", "score": 0.41, "via": "1-hop: Centre Pivot 2
-related_to-> Beans" } ] }
  ]
}
```


Step 4 - LLM output (forced tool call)


```
[
  { "capture_id": "0192f7b2-3a10-7c44-8e21-6b5d9f0a1c77",
"event_index": 0,
```

```
    "summary": "Bean planting completed (final 200 ha)",
    "actions": [
      { "type": "update_attributes", "node_id": "c3a1e0f2-7b6d-4c5e-
9a8b-1d2e3f4a5b10",
        "increment": { "planted_ha": 200 }, "set": { "phase":
"planted" },
        "confidence": 0.92, "evidence": "planting the last 200
hectares of beans" },
      { "type": "append_note", "node_id": "d4b2f1e3-8c7e-4d6f-8b9c-
2e3f4a5b6c20",
        "text": "Planted the final 200 ha of beans. Planting
complete.",
        "tags": ["planting", "milestone"], "confidence": 0.93,
        "evidence": "Just finished planting the last 200 hectares of
beans" },
      { "type": "set_status", "node_id": "c3a1e0f2-7b6d-4c5e-9a8b-
1d2e3f4a5b10",
        "status": "done", "reason": "Planting complete (3,000 / 3,000
ha)",
        "confidence": 0.88, "evidence": "finished planting the last
200 hectares" } ] },
  { "capture_id": "0192f7b2-3a10-7c44-8e21-6b5d9f0a1c77",
"event_index": 1,
    "summary": "Main water pump leaking badly",
    "actions": [
      { "type": "append_note", "node_id": "e5c3a2f4-9d8f-4e7a-9cad-
3f4a5b6c7d30",
        "text": "Main water pump is leaking badly.", "tags": ["leak"],
        "confidence": 0.95, "evidence": "the main water pump is
leaking badly" },
      { "type": "set_status", "node_id": "e5c3a2f4-9d8f-4e7a-9cad-
3f4a5b6c7d30",
        "status": "critical", "reason": "Bad leak on main water supply
(newly planted beans depend on it)",
        "confidence": 0.9, "evidence": "leaking badly" },
      { "type": "create_alert", "node_id": "e5c3a2f4-9d8f-4e7a-9cad-
3f4a5b6c7d30",
        "severity": "critical", "title": "Main water pump leaking
badly", "category": "leak",
        "confidence": 0.9, "evidence": "leaking badly" },
      { "type": "create_task", "node_id": "e5c3a2f4-9d8f-4e7a-9cad-
3f4a5b6c7d30",
        "title": "Inspect and repair main water pump leak",
"priority": "urgent",
        "due_at": null, "assignee_hint": null, "confidence": 0.87,
"evidence": "leaking badly" },
      { "type": "link_nodes", "src_id": "e5c3a2f4-9d8f-4e7a-9cad-
3f4a5b6c7d30",
        "dst_id": "c3a1e0f2-7b6d-4c5e-9a8b-1d2e3f4a5b10", "edge_type":
"affects",
```

```
        "confidence": 0.7, "evidence": "main water pump" } ] }
]
```

Validation passes: all node ids are in the candidate set, `planted_ha` becomes 3,000 which does not exceed `area_ha`, and every confidence is above 0.6. A guard rule would have rejected the increment if it pushed `planted_ha` past `area_ha` and raised an ask_user instead.

Step 5 - Database mutation (one transaction)

```
begin;
select nc_apply_actions(
  p_capture  => '0192f7b2-3a10-7c44-8e21-6b5d9f0a1c77',
  p_actor    => 'ai:claude-sonnet-5-5',
  p_actions  => $json$ [ ...validated actions from Step 4... ]
$json$::jsonb
);
-- inside nc_apply_actions, for this capture:
--   update nodes set attributes = attributes ||
'{"planted_ha":3000,"phase":"planted"}',
--          status='done', status_reason='Planting complete (3,000 /
3,000 ha)', version=version+1
--    where id='c3a1e0f2-...';
--   insert into notes (node_id, capture_id, body_md, tags, author)
values (...), (...);
--   update nodes set status='critical', status_reason='Bad leak on
main water supply ...'
--    where id='e5c3a2f4-...';
--   insert into alerts (node_id, capture_id, severity, category,
title) values (...);
--   insert into tasks  (node_id, capture_id, title, priority) values
(...);
--   insert into edges  (workspace_id, src_id, dst_id, edge_type,
created_by)
--          values (..., 'affects', 'ai') on conflict do nothing;
--   perform nc_rollup('c3a1e0f2-...'); perform
nc_rollup('e5c3a2f4-...');
--   insert into action_log (..., provenance) values (...,
'{"pipeline":"notecad-brain 1.0"}');
update captures set state = 'applied' where id = '0192f7b2-3a10-7c44-
8e21-6b5d9f0a1c77';
commit;
```

Step 6 - Real-time event and UI update

```
{
  "event": "postgres_changes",
  "schema": "public", "table": "nodes", "eventType": "UPDATE",
  "new": {
    "id": "e5c3a2f4-9d8f-4e7a-9cad-3f4a5b6c7d30",
    "title": "Main Water Pump",
    "status": "critical", "status_reason": "Bad leak on main water
supply (newly planted beans depend on it)",
```

```
    "open_tasks": 1, "open_alerts": 1, "version": 18
  },
  "commit_timestamp": "2026-10-14T14:42:12.418Z"
}
```

View	What the user sees within ~2-4 s
Floating widget / Live Activity	"Filed 2 items: Beans - planting complete; Main Water Pump - urgent leak (task created)" with Undo
Mind Map	`Beans` shows a green check and a full 3,000/3,000 ha ring; `Main Water Pump` turns red and pulses with badge 2; `Water` and `Infrastructure` get red echo dots; a new dashed `affects` link runs from the pump to Beans
Folder Tree	Infrastructure > Water > Main Water Pump row turns red with an alert icon; Crops > Beans shows "Planted"
Notepad	New log lines appear at the top of both nodes' notes with time, GPS pin and a play button for the original audio
Push	Critical alert to all workspace members (e.g. the farm manager)

Step 7 - Learning side-effects

- `profile_facts['crops.beans.planted_ha'] = 3000` and the planting season close date are stored.
- The nightly job notes "planting finished 14 Oct" against last year's date for the seasonal calendar.
- If the user had corrected a route (e.g. moved the note to Borehole 2), the phrase "main water pump" would gain an alias weight on Borehole 2 for next time.

6. Phased Development Roadmap

Phase	Scope	Deliverables	Exit criteria
Phase 1 - Core brain (MVP, ~6 weeks)	Database, routing, one capture surface, one view	Supabase migrations (Section 4); brain worker with STT + segment + retrieve + route + validate + apply; JSON schema package; PWA with record button, Folder Tree and simple Mind Map; onboarding	20 real farm voice notes routed with >= 85% correct targets; no lost or duplicated captures in a forced-offline test

Phase	Scope	Deliverables	Exit criteria
		interview; Inbox for ask_user	
Phase 2 - Views & status (~6 weeks)	Full multi-view canvas and status system	React Flow mind map with d3-force and per-lens layouts; Tiptap + Yjs notepad; rollup trigger; badges, echo dots, decay; Realtime wiring; live partial transcript glow; design-system theme	Change in any view visible in all others < 1 s; Lighthouse PWA pass; reduced-motion respected
Phase 3 - Mobile, widget, offline (~8 weeks)	Capacitor apps and OS capture surfaces	Android bubble service, QS tile, Glance widget; iOS Control + AudioRecordingIntent + Live Activity + widgets; Tauri desktop tray/hotkey; SQLite outbox + tus + WorkManager/BG tasks; PowerSync replica; whisper.cpp provisional pass; tldraw whiteboard	One-tap capture from lock screen (iOS) and overlay (Android); 48 h offline field test with zero loss
Phase 4 - Platform layer & learning (~8 weeks)	Platform services and long-term memory	Ask NoteCad via the assistant fabric; NoteCad MCP server; single sign-on; Edge AI Node offline / sovereign mode; opt-in sensor bindings; routines miner + nightly consolidation; starter packs; Afrikaans tuning; sharing and roles; POPIA tooling (export, delete, retention)	Pilot with 10+ users from different fields; >= 90% routing accuracy; each pilot workspace built only from its owner's onboarding

Each phase ends with a tagged release, an ADR set in `docs/adr`, and the document metadata updated.

7. Design System & Platform Services

NoteCad is a personal product: every person who uses it gets their own private workspace, vocabulary and memory. What it shares with any host platform is a thin **platform layer**: a design system, a document standard and a few optional services. Other projects are deliberately not wired in; a user can connect their own sources later through MCP or sensor bindings (3.6).

7.1 Design system

Design element	Source	How NoteCad uses it
Colour tokens	`--brand-primary` `#6366f1`, `--brand-accent` `#14b8a6`, `--brand-info` `#06b6d4`, surfaces `#0f1115` / `#171a21` / `#1e222b`, text `#e6e8ec` / `#8a919e`	All four views, the widget and the Live Activity; status colours add a small `--nc-*` extension (1.2)
Brand gradient	`--brand-gradient` (135deg primary to accent)	Floating capture button, recording ring, app icon background, onboarding hero
Semantic colours	`--color-ok`, `--color-warn`, `--color-danger` (+ `-bg` variants)	Toasts, Inbox cards, alert banners
Typography	Roboto (UI), Roboto Mono (code / data); scale 11-32 px; weights 400-700	Bubble labels, notepad, transcripts; mono for attributes and timestamps
Spacing & motion	4 px spacing scale; `fade-up` animation	Layout grid; node enter animations (disabled under reduced-motion)
Components	Buttons (primary / secondary / ghost), inputs, cards, badges, tables, logo mark	Settings, Inbox, task lists, folder tree rows
Logos & icons	Product wordmark, app icons 192 / 512, apple-touch icon, social card	App icon, splash screen, PWA manifest, store listings, “Ask NoteCad” button
Document template	A house Word / PDF template	“Export to Word / PDF” for any node or report is rendered on this template
Metadata	Document ID, version and pipeline name	Written into `action_log.provenance`, file metadata of every export, and the app’s About screen
Voice	Plain, evidence-first wording	Onboarding copy, empty states, store text

7.2 Platform services NoteCad can connect to (all optional)

Service	What it provides	Role in NoteCad
Assistant fabric	A model gateway that handles text, code, images, vision and web, with the model chosen per task	Runs routing and "Ask NoteCad"; model choice stays vendor-neutral and managed centrally
Edge AI Node	Offline-first on-premises AI box: local models, data in-house, cloud optional	Offline and sovereign mode (2.3) for users who need it
On-device privacy pattern	Audio, vision and sensor processing that never uploads	Basis for the on-device provisional transcript and its privacy wording ("processed on this device")
Single sign-on gateway	OpenID Connect sign-in with Google and 2FA	Sign-in for NoteCad accounts. Without a gateway, NoteCad uses Supabase Auth with Google + 2FA
Product website	Product page, privacy, cookies, terms, security.txt, llms.txt	Legal pages linked from the app, support links; llms.txt entry so AI assistants describe NoteCad correctly
Link registry	Machine-readable list of every official link (links.json / .yml)	Single source for every link shown in the app (About, Help, Legal)
Support channels	Support e-mail and chat	In-app Help and feedback route
Integration services	AI integration, AIoT & edge, predictive maintenance	Upgrade path for users who want sensors or custom integrations feeding their NoteCad

7.3 Personalisation model (one person, one workspace)

- **Own workspace by default.** Sign-up creates a private workspace; sharing with family, staff or a team is an explicit invite with roles (owner, editor, viewer).
- **Own ontology.** The spine comes from the person's onboarding conversation and grows from their notes. Starter packs are suggestions only.
- **Own vocabulary.** Aliases, keyterms and languages are learned per person (e.g. "die groot pomp" for one user, "P1" for another).
- **Own memory.** Profile facts, routines and summaries never mix across workspaces; RAG and MCP access are scoped by workspace token.
- **Own look (within the design system).** Light or dark, density, bubble size and which capture surfaces are enabled.

- **Own data.** Export everything (JSON + Word/PDF) or delete the workspace at any time (POPIA).

Repository: the code lives in a private repository. Secrets, `.env` files and user data are never committed.

8. Security, Privacy (POPIA) & Risks

- **Personal information:** voice recordings, GPS and staff names are personal information under South Africa's POPIA. Store in a South African or EU region where possible, encrypt at rest, record a lawful purpose, and give users export and delete for their workspace.
- **Retention:** keep raw audio 90 days by default (configurable), transcripts and notes until the user deletes them.
- **Least privilege:** RLS on every table; the brain uses the service role only inside the worker; MCP tools are scoped per workspace token.
- **Prompt-injection hygiene:** transcripts and sensor data are data, not instructions; the router only emits schema-valid actions, and the validator rejects anything outside the candidate set.
- **Undo everywhere:** every AI action is in `action_log` and reversible from the widget toast or the node history.

Risk	Impact	Mitigation
Mis-routing in early use	User trust drops	Confidence gates, Inbox ask-back, one-tap correction that trains aliases
Afrikaans / code-switching STT quality	Wrong facts	Whisper path for `af`, keyterms, show transcript with audio playback
iOS background limits	Missed captures	AudioRecordingIntent + Live Activity; Control and widget fall-backs
Android OEM battery killers	Bubble disappears	Foreground service + onboarding guide for battery-optimisation exemptions
Third-party licence cost (tldraw)	Budget	Licence in Phase 3 or switch to Excalidraw
Cloud outage / no internet	No processing	Offline mode via an Edge AI Node; outbox guarantees no loss

9. Next Prompts (build piece by piece)

Paste these one at a time after this blueprint. They reference the names used above, so Claude's code will line up across steps.

1. **Step 1 - Database + AI router:** “Using the NoteCad blueprint (NOTECAD-BP-001), write the complete Supabase SQL migrations from Section 4 (tables, enums, indexes, RLS, the nc_rollup trigger and the nc_apply_actions function), the JSON Schema in packages/schema/actions.schema.json, and the Python FastAPI brain worker (pgmq consumer -> STT -> segmentation -> hybrid retrieval -> Claude tool call with the Section 3.4 system prompt -> jsonschema validation -> nc_apply_actions). Include a seed script for the farming workspace (cows, sheep, 3,000 ha beans, vehicles, water infrastructure) and pytest golden tests for the sheep/tractor and beans/pump notes.”
2. **Step 2 - Interactive prototype:** “Build an interactive React prototype of the NoteCad Mind Map + Folder Tree + Notepad using the design system (background #0f1115, surface #171a21, primary #6366f1, accent #14b8a6, brand gradient primary-to-accent, Roboto) and the Section 1.2 status system. Seed it with the farm example as one sample user. Clicking a bubble selects it in all views. Add a ‘Simulate voice note’ box: typing the beans/pump sentence plays the Section 5 actions with animations (bubble turns red, badge appears, new affects link, notes appended).”
3. **Step 3 - Mobile wrapper + floating recorder:** “Write the Capacitor mobile wrapper for NoteCad: the Android BubbleService (overlay, microphone foreground service, hold-to-talk, Opus recording), the iOS AudioRecordingIntent + ControlWidget + Live Activity, the SQLite outbox and tus upload with WorkManager / background URLSession, and the shared TypeScript capture plugin API used by the web app.”
4. **Step 4 - Platform layer:** “Write the NoteCad MCP server (search_nodes, get_node, append_note, create_task, list_alerts, ask_workspace) scoped per user workspace, the Ask NoteCad panel that answers from the user’s own nodes via RAG, the single sign-on flow (Google + 2FA), the export-to-Word feature on the house document template, and the Edge AI Node offline profile.”

Appendix A - Document Metadata

```
================================================================================
NOTECAD - VOICE-FIRST AI WORKSPACE - MASTER BLUEPRINT
Document ID: NOTECAD-BP-001 | Version: 1.1 | Updated: 2026-10-14
Classification: CONFIDENTIAL - internal product architecture
================================================================================
```

Appendix B - Glossary

Term	Meaning
Node	Any item in NoteCad: category, asset, animal, field, place, person, topic
Channel node	A child folder such as Health Alerts or Maintenance whose status mirrors onto its parent
Spine	Top two levels of the node tree, always sent to the router
Capture	One voice recording (or IoT event) with a

Term	Meaning
	device-generated UUIDv7
Echo dot	Small indicator on an ancestor showing the worst status below it
Outbox	On-device queue that guarantees captures survive offline periods
Edge AI Node	Optional offline-first on-premises AI box running local STT and models